CERTIK

Preliminary Comments

# Exocore (Smart Contracts)

CertiK Assessed on Aug 22nd, 2024

CertiK Assessed on Aug 22nd, 2024

# Exocore (Smart Contracts)

These preliminary comments were prepared by CertiK, the leader in Web3.0 security.

## Executive Summary

| TYPES | ECOSYSTEM | METHODS |
|---|---|---|
| DeFi | Ethereum (ETH) | Formal Verification, Manual Review, Static Analysis |
| **LANGUAGE** | **TIMELINE** | **KEY COMPONENTS** |
| Solidity | Delivered on 08/22/2024 | N/A |
| **CODEBASE** | | **COMMITS** |
| exocore-contracts<br>View All in Codebase Page | | 61ad95f90de462ff0cec1e9ccf5372730be1c603<br>View All in Codebase Page |

## Vulnerability Summary

| 20 | 0 | 0 | 0 | 0 | 0 | 20 |
|---|---|---|---|---|---|---|
| Total Findings | Resolved | Mitigated | Partially Resolved | Acknowledged | Declined | Pending |

| | | | |
|---|---|---|---|
| 0 | Critical | | Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks. |
| 3 | Major | 3 Pending | Major risks can include centralization issues and logical errors. Under specific circumstances, these major risks can lead to loss of funds and/or control of the project. |
| 1 | Medium | 1 Pending | Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform. |
| 10 | Minor | 10 Pending | Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions. |
| 1 | Informational | 1 Pending | Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code. |
| 5 | Discussion | 5 Pending | The impact of the issue is yet to be determined, hence requires further clarifications from the project team. |

# TABLE OF CONTENTS | EXOCORE (SMART CONTRACTS)


## Summary

- Executive Summary
- Vulnerability Summary
- Codebase
- Audit Scope
- Approach & Methods

## Review Notes

- Overview
  - ExocoreGateway
  - ClientChainGateway
  - Vault
  - ExoCapsule
  - Bootstrap
- Reviewed Items
  - Manual Review
  - Security Checklist
- External Dependencies
  - Addresses
- Privileged Functions

## Findings

- EGE-01 : Tokens May Not Be Added To Whitelist on Client Chains
- ENB-02 : Centralized Control of Contract Upgrade
- GLOBAL-01 : Centralization Related Risks
- EGE-02 : Unauthorized Bootstrap Request May Lead to DoS Situation
- BEN-01 : Lack of Initialization of OApp For `Bootstrap`
- BEN-02 : Registered Validators Are Not Sorted
- BEN-03 : Locked Blockchain Native Tokens
- ECE-01 : Unreachable Function
- ECE-02 : Missing initialization of `ReentrancyGuardUpgradeable`
- EGE-03 : Potential Improper Refund Address Handling for Fee Overpayments in Cross-Chain Messages
- EGE-05 : Unresolved `TODO` Comment

ENB-03 : Lack of Storage Gap Or NameSpaced Storage Layout in Upgradeable Contract

OAS-02 : Potential Unintended Native Token Transfer

VEN-01 : Incompatibility with Deflationary Tokens

BEN-04 : Improper Use of `msg.sender` in `_delegateTo` Function of `Bootstrap` Contract

EGE-04 : Discussion on Processing `REQUEST_DEPOSIT` Request in `ExocoreGateway`

GSE-01 : Discussion on Inability To Remove Whitelisted Tokens

NRC-01 : Discussion on Partial Withdrawal Request Omission in `processBeaconChainWithdrawal` Function

OAS-01 : Discussion on `_lzSend` Function in `OAppSenderUpgradeable` Contract

VEN-02 : Discussion on Source of Vault Rewards

## Optimizations

ENB-01 : Inefficient Memory Parameter

## Appendix

## Disclaimer

# CODEBASE | EXOCORE (SMART CONTRACTS)

## Repository

exocore-contracts

## Commit

61ad95f90de462ff0cec1e9ccf5372730be1c603

# AUDIT SCOPE | EXOCORE (SMART CONTRACTS)

28 files audited ● 16 files with Pending findings ● 12 files without findings

| ID | Repo | File | SHA256 Checksum |
|---|---|---|---|
| ● BRC | ExocoreNetwork/exocore-contracts | src/core/BaseRestakingController.sol | c45c839785365f1f4a1cd6fd131c6de3ef405933d6ae9686a4377e7b830349d0 |
| ● BEN | ExocoreNetwork/exocore-contracts | src/core/Bootstrap.sol | 258a240e75b28f3d0e26e49952796938d81828433a73f2bcd0a2c4c3a5c90253 |
| ● BLR | ExocoreNetwork/exocore-contracts | src/core/BootstrapLzReceiver.sol | ce8efab117a9c27869743de916467a328693abaabbc7eb4308bef280ce9e7287 |
| ● CCG | ExocoreNetwork/exocore-contracts | src/core/ClientChainGateway.sol | 01af3e392c50ade3645776f6701d67ef84428408ace9581a066046ab8bfdef9f |
| ● CGL | ExocoreNetwork/exocore-contracts | src/core/ClientGatewayLzReceiver.sol | 96fe2864b95b03c208e86d365a9cffdc5bfa12c56449082262232c8c6e6148d5 |
| ● CPA | ExocoreNetwork/exocore-contracts | src/core/CustomProxyAdmin.sol | cdb448b707764aa74b828ad333b1eb153bee5e4f6b42b417c8d7f8daaecb744e |
| ● ECE | ExocoreNetwork/exocore-contracts | src/core/ExoCapsule.sol | 0c662b71d9bd35a76e6421cac626b98d0541bdbe96ddb9486321291f442c4b16 |
| ● EGE | ExocoreNetwork/exocore-contracts | src/core/ExocoreGateway.sol | 5b09bd335c08b9cc4258e67ee0116f8bed686550a3b2706c276973977d6d8d18 |
| ● LST | ExocoreNetwork/exocore-contracts | src/core/LSTRestakingController.sol | 410ba8645685850331b44316103e64e61dacb1219d8f4c9f6d6db49be7d9529a |
| ● NRC | ExocoreNetwork/exocore-contracts | src/core/NativeRestakingController.sol | 0e87a97954f9fb2711ccc45211dd5de0045b190baf9ee289aad727a009f9fa8c |
| ● VEN | ExocoreNetwork/exocore-contracts | src/core/Vault.sol | 2d9880bf39233632c55e0eb04648b421788bc9d37ce23d62586a7e3a60dfc67b |
| ● OAC | ExocoreNetwork/exocore-contracts | src/lzApp/OAppCoreUpgradeable.sol | 9c027be2ca883155f6ed00bacf8d984e8c4c6a69c7c5ec4954e106e649f0fb04 |
| ● OAR | ExocoreNetwork/exocore-contracts | src/lzApp/OAppReceiverUpgradeable.sol | d2a299ba83592af1f151367f136b49a387bbc5c74bbe1f675c8bea1e800d58b2 |

| ID | Repo | File | SHA256 Checksum |
|---|---|---|---|
| OAS | ExocoreNetwork/exocore-contracts | src/lzApp/OAppSenderUpgradeable.sol | bae9ea93be84041e7a336a8100d928be946962e4df901cf7ac91a077cec2bf28 |
| OAU | ExocoreNetwork/exocore-contracts | src/lzApp/OAppUpgradeable.sol | 923473f61542e865c3ed50b51c5345b79d24dc8ebb3f8016bb1c9cfdbee5a95c |
| GSE | ExocoreNetwork/exocore-contracts | src/storage/GatewayStorage.sol | adca58f125cd28d91fdd7b8f5c01c393120bb90ef38fc145867dfc5da16f0e57 |
| BPB | ExocoreNetwork/exocore-contracts | src/core/BeaconProxyBytecode.sol | 6d30fccc5534addf33971104fb2ff44903a50d8f9dbf4d0296a7517518537dcb |
| BCP | ExocoreNetwork/exocore-contracts | src/libraries/BeaconChainProofs.sol | 15cace49989f5d8b09f376edd3018f1378c5a3f1335e5299e7259ce41450f01e |
| EEN | ExocoreNetwork/exocore-contracts | src/libraries/Endian.sol | 50355617ec503fa537c3a45a354ad5f6de8975127c4f11ce168a04f87a35ff8a |
| ERR | ExocoreNetwork/exocore-contracts | src/libraries/Errors.sol | 4cea2e6ab48f60868df850e2856fe4851e91edf243485f54ce2d8a07435e1707 |
| MEN | ExocoreNetwork/exocore-contracts | src/libraries/Merkle.sol | 87f1e5632442d8f081219879c054084d4c1446e2bb081b576f444e6c07400c42 |
| VCE | ExocoreNetwork/exocore-contracts | src/libraries/ValidatorContainer.sol | b85aaa5e1f64d9744390f40ede45b31cf989ebb979c6ae014b5c5e0ed6f0dfb9 |
| WCE | ExocoreNetwork/exocore-contracts | src/libraries/WithdrawalContainer.sol | 7937d59a0c263cdb77ab3eaa4362b50c4a2c87cbc3e14e5c3cdf8bdede803c17 |
| BSE | ExocoreNetwork/exocore-contracts | src/storage/BootstrapStorage.sol | 5aacbf7fa98905cd8a45fd8b5fa0e8d2d8b945e6acdb29325d09fca15243e54e |
| CCS | ExocoreNetwork/exocore-contracts | src/storage/ClientChainGatewayStorage.sol | a10c44b5548e7ffec75a958c2f480bc3e9276e0a9f6ab1bc1b4afd97d7841dcd |
| ECS | ExocoreNetwork/exocore-contracts | src/storage/ExoCapsuleStorage.sol | 5df3f90fed9e3c8a8ef6ac51b1f8fc5163f8bd311554fdc899eac14dfffb0f6e |
| EGS | ExocoreNetwork/exocore-contracts | src/storage/ExocoreGatewayStorage.sol | dd85696b4423ac011ae5f0a4a391af53b32bfa73bc15a668e0e39ceac34a7947 |
| VSE | ExocoreNetwork/exocore-contracts | src/storage/VaultStorage.sol | 9bb016bd4b5abca6662dc8ad39fdd80fe7b1d3bd4b958527ef29e2319dd48ee2 |

# APPROACH & METHODS | EXOCORE (SMART CONTRACTS)

This report has been prepared for Exocore to discover issues and vulnerabilities in the source code of the Exocore (Smart Contracts) project as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Formal Verification, Manual Review, and Static Analysis techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

The security assessment resulted in findings that ranged from critical to informational. We recommend addressing these findings to ensure a high level of security standards and industry practices. We suggest recommendations that could better serve the project from the security perspective:

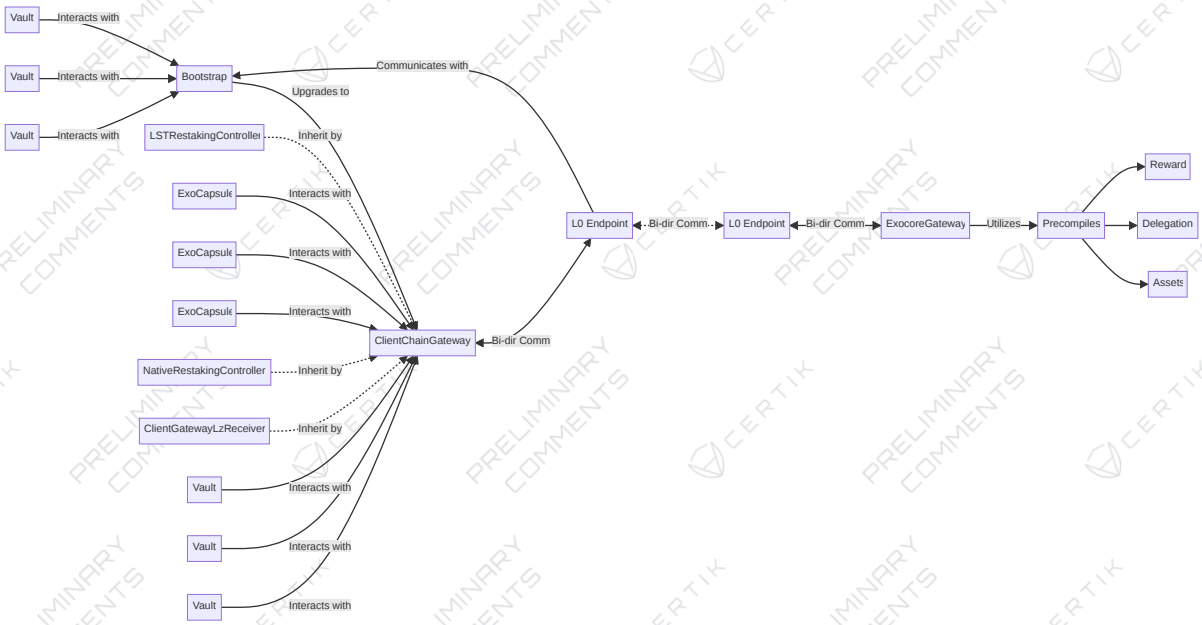
- Testing the smart contracts against both common and uncommon attack vectors;
- Enhance general coding practices for better structures of source codes;
- Add enough unit tests to cover the possible use cases;
- Provide more comments per each function for readability, especially contracts that are verified in public;
- Provide more transparency on privileged activities once the protocol is live.

# REVIEW NOTES | EXOCORE (SMART CONTRACTS)

## Overview

**Exocore** is the first layer-one omnichain restaking protocol. The Exocore smart contracts are deployed on both Exocore and the target client chains, which facilitate assets deposit and withdrawal, cross-chain communication, and restaking operations for native assets and liquid staking tokens (LSTs).

### Architecture Diagram




This diagram provides a detailed overview of Exocore's contract architecture, highlighting the interactions between the client chain gateway and the Exocore gateway.

The main components of the Exocore contracts include:

### ExocoreGateway

The ExocoreGateway contract handles cross-chain messaging and interactions with the Exocore chain. This contract provides functionalities such as deposits, delegations, undelegations, and the withdrawal of principal and rewards. It facilitates cross-chain communication through the LayerZero protocol.

- ExocoreGateway
- ExocoreGatewayStorage
- GatewayStorage

### ClientChainGateway

This is the entry point where client chain users make requests to the Exocore validator set, as well as the endpoint that receives cross-chain messages from the Exocore validator set.

### ClientGatewayLzReceiver

This contract receives messages and processes the requests over LayerZero from the Exocore Gateway.

### BaseRestakingController

The BaseRestakingController contract is an abstract contract. It primarily handles the fundamental logic related to restaking, including declarations, delegations, undelegations, and messaging with the Exocore chain.

### LSTRestakingController

The LSTRestakingController contract an abstract contract, which provides functionalities for users to deposit tokens, withdraw principal from Exocore, and withdraw rewards from Exocore. It primarily handles operations related to Liquid Staking Tokens (LST).

### NativeRestakingController

The NativeRestakingController contract is an abstract contract, which primarily handles the logic for native Ethereum restaking. It includes functionalities such as staking, creating ExoCapsule contracts, and managing deposits and partial/full withdrawals for Beacon Chain validators.

## Vault

This is where user funds are taken into custody and managed. Within Vault, user balance is updated periodically by Exocore validator set through cross-chain message to reveal user's real position (after slashing, rewarding and other impact). Vault is created through Bootstrap and ClientChainGateway contracts. Users can withdraw from Vault based on grant from the gateway. Every specific asset should have a standalone Vault.

- Vault
- VaultStorage

## ExoCapsule

The ExoCapsule contract is designed to handle specific operations related to native Ethereum restaking tokens. It provides functionalities for depositing and withdrawing these tokens, ensuring secure and efficient management of virtual staked assets.

- ExoCapsule
- ExoCapsuleStorage

## Bootstrap

The Bootstrap smart contract is designed to manage token whitelisting, operator registration, deposits, and withdrawals. It ensures that operations are locked during specific periods (before mainnet/testnet launch) for security. This contract is independent and upgrades itself to ClientChainGateway after the mainnet/testnet launch.

**BootstrapLzReceiver**

This contract receives messages from the Exocore chain for bootstrapping.

## Reviewed Items

The audit is based on the following documentations from the Exocore team

- Smart contract specifications: https://exocore.notion.site/4176fc81e0aa4c0fb1469b0810c5c417?v=61224acf6e76410287cc5dca50b6d257
- Official Docs: https://docs.exocore.network/
- Exocore Github Docs: https://github.com/ExocoreNetwork/exocore-contracts/tree/main/docs

### Manual Review

`storage` :

- Verify that shared storage module is correctly defined and used.
- Verify the implementation of storage gaps for future upgradeability.

`lzApp` :

- Verify that all abstract contracts correctly override the OApp's basic configurations from LayerZero.
- Verify that all functions that update sensitive state variables are restricted to privileged roles.
- Verify that the `_quote` , `lzReceive` , `_lzReceive` , and `_lzSend` functions are correctly implemented.

`ExocoreGateway` :

- Verify that the `onlyCalledFromThis` modifier is used for internal function calls.
- Verify that pausable functionality is correctly implemented and working with the `pause()` and `unpause()` functions.
- Verify that external contract interactions ensure `ASSETS_CONTRACT` , `CLAIM_REWARD_CONTRACT` , and `DELEGATION_CONTRACT` addresses are correctly set and used.
- Verify that `_sendInterchainMsg()` is correctly constructing and sending interchain messages.
- Verify that deposit and withdrawal functions with `requestDeposit()` , `requestWithdrawPrincipal()` , and `requestWithdrawReward()` are correctly handling deposits and withdrawals.
- Verify that delegation functions with `requestDelegateTo()` , `requestUndelegateFrom()` , and `requestDepositThenDelegateTo()` are correctly handling delegation operations.
- Verify that operator association functions with `associateOperatorWithEVMStaker()` , `dissociateOperatorFromEVMStaker()` , `requestAssociateOperatorWithStaker()` , and `requestDissociateOperatorFromStaker()` are correctly handling operator associations.
- Verify that LayerZero integration with `_lzReceive()` is correctly processing incoming LayerZero messages.

`Bootstrap`:

- Verify that the `beforeLocked` modifier correctly restricts operations based on the contract's timeline.
- Verify that the `registerValidator` function correctly validates and registers validators.
- Verify that the `deposit` function correctly transfers tokens to the vault and updates the deposit records.
- Verify that the `withdrawPrincipalFromExocore` function correctly validates and processes withdrawals.
- Verify that the `claim` function correctly processes token claims.
- Verify that the `delegateTo` / `undelegateFrom` functions correctly validates and processes delegations/undelegations.
- Verify that the `markBootstrapped` function correctly validates and updates the bootstrapped state and upgrades the contract implementation.

`ExoCapsule`:

- Verify that the `verifyDepositProof` / `verifyWithdrawalProof` function correctly validates the validator container and updates the validator's status and deposit/withdrawal amounts.
- Verify that the `withdraw` function correctly handles withdrawals, checks the validity of the recipient address and amount, and updates the `withdrawableBalance`.
- Verify that the `capsuleWithdrawalCredentials` function correctly constructs the withdrawal credentials.

`Vault`:

- Verify that the `withdraw` function ensures the `amount` does not exceed the `withdrawableBalances`, correctly updates the `withdrawableBalances`, and transfers the `underlyingToken` to the `recipient`.
- Verify that the `deposit` function correctly transfers the `underlyingToken` from the `depositor` to the contract and accurately updates the `totalDepositedPrincipalAmount` for the `depositor`.

`BaseRestakingController`:

- Verify only the allowlisted tokens can be used in the contract.
- Verify that the `claim` function correctly distinguishes between ETH and ERC20 token withdrawals and the withdrawable balance should be decreased by the correct amount.
- Verify that the `delegateTo` function correctly encodes and sends a `REQUEST_DELEGATE_TO` message to the Exocore chain and verify the packing and encoding of arguments in `delegateTo` are correct.
- Verify that the `undelegateFrom` function correctly encodes and sends a `REQUEST_UNDELEGATE_FROM` message to the Exocore chain and verify the packing and encoding of arguments are correct.

`LSTRestakingController`:

- Verify only the allowlisted tokens can be used in the contract.
- Verify the functions which send message are marked as `payable` because the contract requires the `msg.sender` to pay the message fees.

- Verify that the `deposit` function correctly interacts with the corresponding `IVault` instance to deposit tokens and sends the expected message.
- Verify that the `withdrawPrincipalFromExocore` function properly encodes and sends a `REQUEST_WITHDRAW_PRINCIPAL_FROM_EXOCORE` message, ensuring accurate packing of arguments.
- Verify that the `withdrawRewardFromExocore` function sends a `REQUEST_WITHDRAW_REWARD_FROM_EXOCORE` message accurately, with correctly encoded and packed parameters.
- Verify that the `depositThenDelegateTo` function effectively manages the combined operations of depositing and delegating by correctly processing the request.

`NativeRestakingController`:

- Verify that the exact 32 ETH deposit to the Ethereum beacon chain is executed correctly using the validator's public key, signature, and deposit data root, and that the capsule's withdrawal credentials are used.
- Verify that the `createExoCapsule` function checks for an existing capsule and reverts if one is already associated with the sender and ensure the that the new capsule is initialized correctly.
- Verify that the capsule associated with the sender is correctly retrieved in the `depositBeaconChainValidator` function and used to verify the deposit proof from the beacon chain and ensure the deposit value is correctly extracted from the proof verification process.
- Verify that the `processBeaconChainWithdrawal` function correctly processes full withdrawals by sending a `REQUEST_WITHDRAW_PRINCIPAL_FROM_EXOCORE` message, and that partial withdrawals are handled appropriately without sending this message.

`ClientGatewayLzReceiver`:

- Verify that the `_lzReceive` function correctly decodes the action from the payload and routes the execution based on whether it's a response or a request, using properly whitelisted function selectors and verify that the nonce is correctly managed and updated to prevent replay attacks.
- Verify that the `_handleResponse` function accurately determines the type of response (basic or balance update) and processes it accordingly and ensure that state updates for assets or rewards are handled correctly based on the success of the response, particularly ensuring correct behavior for deposit and withdrawal operations and verify that the function properly cleans up the cache after processing the response to prevent stale data issues.
- Verify that the `_updatePrincipalAssetState` function interacts correctly with the appropriate contract (`IExoCapsule` or `IVault`) based on the token type and performs the necessary state updates.
- Verify that the `afterReceiveAddWhitelistTokensRequest` function correctly whitelists tokens, avoids duplicating records and creates vaults where necessary.

`ClientChainGateway`:

- Verify that the `_clearBootstrapData` function clears all specified state variables and sensitive data to reset the contract state and prevent any residual data misuse.
- Verify that the `quote` function accurately calculates the native fee for messaging based on the provided message and options.

`CustomProxyAdmin`:

- Verify that the `initialize` function correctly sets the `bootstrapper` state variable to the `newBootstrapper` address and only the contract owner can call this function.
- Verify that the `changeImplementation` function sets the `bootstrapper` state variable to zero after execution and ensure that only the specified `bootstrapper` can call this function, and it must also be the proxy itself.

### Security Checklist

- Ensure that only authorized addresses and accounts can modify critical state variables.
- Ensure that all the messages and requests have been validated before execution.
- Ensure that the modifier is properly defined and allows only authorized addresses to execute these critical functions.
- Ensure that all the necessary dependencies are identified, and notify the team that all these dependency contracts or accounts must be thoroughly reviewed and audited before being used.
- Ensure that all the libraries are correctly used.
- Ensure that no integer overflow/underflow and rounding issues occur.
- Ensure that all external calls within the function are performed after all state changes to follow the "checks-effects-interactions" pattern.
- Ensure that the balance updates are performed correctly both in the ClientChainGateway, Vault, Exocapsule contracts.
- Ensure that all the corner cases are properly documented and that there are mechanisms in place to handle these cases.
- Ensure that the alignment of storage layout is correct.
- Ensure that there is no Denial of Service attacks within this project.
- Ensure that events are emitted for important state changes to provide transparency of on-chain activities.

## External Dependencies

In **Exocore**, the module inherits or uses a few of the depending injection contracts or addresses to fulfill the need of its business logic. The scope of the audit treats third party entities as black boxes and assume their functional correctness. However, in the real world, third parties can be compromised and this may lead to lost or stolen assets.

### Addresses

The following addresses interact at some point with specified contracts, making them an external dependency. All of following values are initialized either at deploy time or by specific functions in smart contracts.

**BaseRestakingController**:

- `capsule`, `vault`

**Bootstrap**:

- `vault`, `customProxyAdmin`

**ClientGatewayLzReceiver**:

- `vault`, `capsule`

**CustomProxyAdmin**:

- `proxy`

**ExoCapsule**:

- `recipient`

**ExocoreGateway**:

- `ASSETS_CONTRACT`, `DELEGATION_CONTRACT`, `CLAIM_REWARD_CONTRACT`, `DELEGATION_CONTRACT`

We assume these contracts or addresses are valid and non-vulnerable actors and implementing proper logic to collaborate with the current project.

Also, the following library/contract are considered as the third-party dependencies:

- `@openzeppelin/contracts-upgradeable/`
- `@openzeppelin/contracts/`
- `@layerzero-v2/`

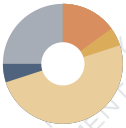
## Privileged Functions

In the **Exocore** project, the privileged roles are adopted to ensure the dynamic runtime updates of the project, which are specified in the following finding: `Centralization Related Risks`.

The advantage of those privileged roles in the codebase is that the client reserves the ability to adjust the protocol according to the runtime required to best serve the community. It is also worth noting the potential drawbacks of these functions, which should be clearly stated through the client's action/plan. Additionally, if the private keys of the privileged accounts are compromised, it could lead to devastating consequences for the project.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Any plan to invoke the aforementioned functions should be also considered to move to the execution queue of the Timelock contract.

# FINDINGS | EXOCORE (SMART CONTRACTS)



| 20 | 0 | 3 | 1 | 10 | 1 | 5 |
|---|---|---|---|---|---|---|
| Total Findings | Critical | Major | Medium | Minor | Informational | Discussion |

This report has been prepared to discover issues and vulnerabilities for Exocore (Smart Contracts) . Through this audit, we have uncovered 20 issues ranging from different severity levels. Utilizing the techniques of Formal Verification, Manual Review & Static Analysis to complement rigorous manual code reviews, we discovered the following findings:

| ID | Title | Category | Severity | Status |
|---|---|---|---|---|
| EGE-01 | Tokens May Not Be Added To Whitelist On Client Chains | Logical Issue | Major | Pending |
| **ENB-02** | **Centralized Control Of Contract Upgrade** | **Centralization** | **Major** | **Pending** |
| **GLOBAL-01** | **Centralization Related Risks** | **Centralization** | **Major** | **Pending** |
| EGE-02 | Unauthorized Bootstrap Request May Lead To DoS Situation | Access Control | Medium | Pending |
| BEN-01 | Lack Of Initialization Of OApp For `Bootstrap` | Inconsistency | Minor | Pending |
| BEN-02 | Registered Validators Are Not Sorted | Inconsistency | Minor | Pending |
| BEN-03 | Locked Blockchain Native Tokens | Inconsistency | Minor | Pending |
| ECE-01 | Unreachable Function | Logical Issue | Minor | Pending |
| ECE-02 | Missing Initialization Of `ReentrancyGuardUpgradeable` | Coding Issue | Minor | Pending |
| EGE-03 | Potential Improper Refund Address Handling For Fee Overpayments In Cross-Chain Messages | Logical Issue | Minor | Pending |

| ID | Title | Category | Severity | Status |
|---|---|---|---|---|
| EGE-05 | Unresolved `TODO` Comment | Coding Style, Logical Issue | Minor | Pending |
| ENB-03 | Lack Of Storage Gap Or NameSpaced Storage Layout In Upgradeable Contract | Design Issue | Minor | Pending |
| OAS-02 | Potential Unintended Native Token Transfer | Logical Issue | Minor | Pending |
| VEN-01 | Incompatibility With Deflationary Tokens | Logical Issue | Minor | Pending |
| BEN-04 | Improper Use Of `msg.sender` In `_delegateTo` Function Of `Bootstrap` Contract | Coding Style | Informational | Pending |
| EGE-04 | Discussion On Processing `REQUEST_DEPOSIT` Request In `ExocoreGateway` | Design Issue, Inconsistency | Discussion | Pending |
| GSE-01 | Discussion On Inability To Remove Whitelisted Tokens | Design Issue | Discussion | Pending |
| NRC-01 | Discussion On Partial Withdrawal Request Omission In `processBeaconChainWithdrawal` Function | Logical Issue | Discussion | Pending |
| OAS-01 | Discussion On `_lzSend` Function In `OAppSenderUpgradeable` Contract | Design Issue | Discussion | Pending |
| VEN-02 | Discussion On Source Of Vault Rewards | Design Issue | Discussion | Pending |

# EGE-01 | TOKENS MAY NOT BE ADDED TO WHITELIST ON CLIENT CHAINS

| Category | Severity | Location | Status |
|---|---|---|---|
| Logical Issue | Major | src/core/ExocoreGateway.sol: 218 | Pending |

## Description

The function `addOrUpdateWhitelistTokens` adds or updates tokens and if a token was added, it sends a `REQUEST_ADD_WHITELIST_TOKENS` message to a client chain.

```
218         if (!updated) {
219             _sendInterchainMsg(
220                 clientChainId,
221                 Action.REQUEST_ADD_WHITELIST_TOKENS,
222                 abi.encodePacked(uint8(tokens.length), tokens),
223                 false
224             );
```

However, the `updated` variable corresponds to the last token in the `tokens` array, meaning that if a token earlier in the array was added and the last token in the array was only updated, the `REQUEST_ADD_WHITELIST_TOKENS` will not be sent.

Note that calling `addOrUpdateWhitelistTokens` again with the recently added token will not resolve the issue as that will update the token, so the `!updated` value will be false and the `REQUEST_ADD_WHITELIST_TOKENS` message will not be sent.

## Proof of Concept

The POC based on `test/foundry/unit/ExocoreGateway.t.sol` shows that the `ExocoreGateway` could potentially not send message to client chains.

```
    function test_POC01_AddWhiteListTokens_NotSyncClientChains() public {
        test_Success_AddWhiteListTokens();

        _prepareInputs(2);
        ERC20 restakeToken2 = new ERC20PresetFixedSupply("rest", "rest", 1e34,
exocoreValidatorSet.addr);
        whitelistTokens[0] = bytes32(bytes20(address(restakeToken2)));
        whitelistTokens[1] = bytes32(bytes20(address(restakeToken)));

        decimals[0] = 18;
        tvlLimits[0] = 1e8 ether;
        names[0] = "RestakeToken2";
        metaData[0] = "ERC20 LST token2";

        decimals[1] = 18;
        tvlLimits[1] = 1e8 ether;
        names[1] = "RestakeToken1";
        metaData[1] = "ERC20 LST token1";

        uint256 messageLength = TOKEN_ADDRESS_BYTES_LENGTH * whitelistTokens.length
+ 2;
        uint256 nativeFee = exocoreGateway.quote(clientChainId, new
bytes(messageLength));
        vm.startPrank(exocoreValidatorSet.addr);

        vm.expectEmit(true, true, true, true, address(exocoreGateway));
        emit WhitelistTokenAdded(clientChainId, whitelistTokens[0]);
        emit WhitelistTokenUpdated(clientChainId, whitelistTokens[1]);
        emit MessageSent(GatewayStorage.Action.REQUEST_ADD_WHITELIST_TOKENS,
generateUID(1, false), 1, nativeFee);
        exocoreGateway.addOrUpdateWhitelistTokens{value: nativeFee}(
            clientChainId, whitelistTokens, decimals, tvlLimits, names, metaData
        );
    }
```

Test result:

```
% forge test --mt test_POC01_AddWhiteListTokens_NotSyncClientChains -vv
[⠊] Compiling...
[⠊] Compiling 1 files with 0.8.24
[⠊] Solc 0.8.24 finished in 7.23s
Compiler run successful!

Ran 1 test for test/foundry/unit/ExocoreGateway.t.sol:AddWhitelistTokens
[PASS] test_POC01_AddWhiteListTokens_NotSyncClientChains() (gas: 1256340)
Logs:
  ~~~_sendInterchainMsg~~~

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 8.78ms (1.41ms CPU
time)
```

In the testing, it doesn't call `_sendInterchainMsg` while adding whitelist tokens in the second time.

## Recommendation

It is recommended to send a `REQUEST_ADD_WHITELIST_TOKENS` message if any token was added, instead of just checking the last token.

# ENB-02 | CENTRALIZED CONTROL OF CONTRACT UPGRADE

| Category | Severity | Location | Status |
|---|---|---|---|
| Centralization | Major | src/core/Bootstrap.sol: 32; src/core/ClientChainGateway.sol: 24; src/core/ExoCapsule.sol: 19; src/core/ExocoreGateway.sol: 33; src/core/Vault.sol: 17 | Pending |

## Description

In the contracts listed as below, the role `admin` has the authority to update the implementation contract behind these contracts.

- Bootstrap
- ClientChainGateway
- ExocoreGateway
- ExoCapsule
- Vault

Any compromise to the `admin` account may allow a hacker to take advantage of this authority and change the implementation contract which is pointed by proxy and therefore execute potential malicious functionality in the implementation contract.

## Recommendation

We recommend that the team make efforts to restrict access to the admin of the proxy contract. A strategy of combining a time-lock and a multi-signature (⅔, ⅗) wallet can be used to prevent a single point of failure due to a private key compromise. In addition, the team should be transparent and notify the community in advance whenever they plan to migrate to a new implementation contract.

Here are some feasible short-term and long-term suggestions that would mitigate the potential risk to a different level and suggestions that would permanently fully resolve the risk.

### Short Term:

A combination of a time-lock and a multi signature (⅔, ⅗) wallet mitigate the risk by delaying the sensitive operation and avoiding a single point of key management failure.

- A time-lock with reasonable latency, such as 48 hours, for awareness of privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to a private key compromised;

AND

- A medium/blog link for sharing the time-lock contract and multi-signers addresses information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

### Long Term:

A combination of a time-lock on the contract upgrade operation and a DAO for controlling the upgrade operation mitigate the contract upgrade risk by applying transparency and decentralization.

- A time-lock with reasonable latency, such as 48 hours, for community awareness of privileged operations;
  AND
- Introduction of a DAO, governance, or voting module to increase decentralization, transparency, and user involvement;
  AND
- A medium/blog link for sharing the time-lock contract, multi-signers addresses, and DAO information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

### Permanent:

Renouncing ownership of the `admin` account or removing the upgrade functionality can *fully* resolve the risk.

- Renounce the ownership and never claim back the privileged role;
  OR
- Remove the risky functionality.

*Note: we recommend the project team consider the long-term solution or the permanent solution. The project team shall make a decision based on the current state of their project, timeline, and project resources.*

# GLOBAL-01 | CENTRALIZATION RELATED RISKS

| Category | Severity | Location | Status |
|---|---|---|---|
| Centralization | Major | | Pending |

## Description

In the contract `Bootstrap` the role `_owner` has authority over the functions shown in the list below.

- **`pause()`** : This function is accessible only by the contract owner. It pauses the contract's operations, preventing execution of functions that modify the state, ensuring no further transactions can be processed until it is unpaused.
- **`unpause()`** : Similar to `pause()`, this function is also exclusive to the contract owner. It resumes normal operations of the contract after being paused, allowing state-modifying transactions to proceed.
- **`setSpawnTime(uint256 spawnTime_)`** : Available only to the owner and must be called before the lock time commences. It allows the owner to update the spawn time of the Exocore chain, which is critical for timing-based operations within the network.
- **`setOffsetDuration(uint256 offsetDuration_)`** : This function can only be called by the owner and before the lock period starts. It updates the offset duration which determines the period during which operations are locked before the Exocore chain's spawn time, crucial for managing operational timelines.
- **`addWhitelistTokens(address[] calldata tokens)`** : Executable only by the owner, this function adds new tokens to the whitelist, enabling them for deposit and other operations within the contract. It is essential for expanding the range of tokens supported by the platform.
- **`registerOrUpdateClientChain(uint32 clientChainId, bytes32 peer, uint8 addressLength, string calldata name, string calldata metaInfo, string calldata signatureType)`** : Exclusively callable by the owner, this function registers or updates a client chain's details in the network, which is vital for maintaining an accurate and current network of participant chains.
- **`setPeer(uint32 clientChainId, bytes32 clientChainGateway)`** : Also restricted to the owner, it sets or updates the peer address for designated client chains, facilitating the correct routing and receipt of cross-chain messages.
- **`setClientChainGatewayLogic(address _clientChainGatewayLogic, bytes calldata _clientChainInitializationData)`** : This owner-only function sets a new client chain gateway logic and its initialization data, allowing for updates in logic before the system is fully bootstrapped.

Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and execute potentially devastating actions such as pausing the contract to disrupt normal operations, unpausing it in unfavorable conditions, or

modifying critical parameters such as spawn times and offset durations. Additionally, the hacker could manipulate the whitelist by adding tokens, alter the logic of client chain gateways, or even redirect funds by changing peer addresses.

Additionally, the `Bootstrap` contract inherits the `OwnableUpgradeable` contract from OpenZeppelin, the owner has the following authorities within the contract:

- **`renounceOwnership()`** : Leaves the contract without owner;
- **`transferOwnership()`** : Transfers ownership of the contract to a new account.

---

In the contract `ClientChainGateway` the role `_owner` has authority over the functions shown in the list below.

- **`pause()`** : Accessible only by the contract owner, this function enables the pausing of all contract operations that are susceptible to modifications, effectively freezing the state of the contract to prevent any state-changing transactions until it is explicitly unpaused.
- **`unpause()`** : This function is complementary to `pause()` and can also be executed only by the contract owner. It resumes the contract's operations by lifting the pause state, allowing previously disabled functionalities to become active again.

Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and execute potentially devastating actions such as pausing the contract to disrupt normal operations, unpausing it in unfavorable conditions.

Additionally, the `ClientChainGateway` contract inherits the `OwnableUpgradeable` contract from OpenZeppelin, the owner has the following authorities within the contract:

- **`renounceOwnership()`** : Leaves the contract without owner;
- **`transferOwnership()`** : Transfers ownership of the contract to a new account.

---

In the contract `CustomProxyAdmin` the role `_owner` has authority over the functions shown in the list below.

- **`initialize(address newBootstrapper)`** : This function is a privileged operation accessible only by the contract owner. It sets up the `bootstrapper` address, which is the proxy that is allowed to upgrade itself using this contract. The function ensures that the contract is properly initialized with a valid bootstrapper address to manage its own upgrades securely. This initialization is crucial for setting up the contract's operational context and can only be called once due to the `initializer` modifier, ensuring that the bootstrapper address is securely set during the contract's deployment phase.

Besides, the `CustomProxyAdmin` contract inherits the `ProxyAdmin` contract from OpenZeppelin, the owner has the following authorities within the contract:

- **`changeProxyAdmin(ITransparentUpgradeableProxy proxy, address newAdmin)()`** : Changes the admin of `proxy` to `newAdmin`.

- `upgrade(ITransparentUpgradeableProxy proxy, address implementation)` : Upgrades `proxy` to `implementation` .
- `upgradeAndCall(..)` : Upgrades `proxy` to `implementation` and calls a function on the new implementation.

Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and initiate unauthorized actions, change the proxy admin and upgrade the implementation logic.

Additionally, the `CustomProxyAdmin` contract inherits the `Ownable` contract from OpenZeppelin, the owner has the following authorities within the contract:

- `renounceOwnership()` : Leaves the contract without owner;
- `transferOwnership()` : Transfers ownership of the contract to a new account.

---

In the contract `ExocoreGateway` the role `_owner` has authority over the functions shown in the list below.

- `pause()` : This function allows the contract owner to pause all operations within the contract, effectively halting any state-changing functions to prevent execution in the case of an emergency or discovery of a critical bug.
- `unpause()` : Complementary to `pause()` , this function allows the contract owner to resume operations after they have been paused, restoring the contract's functionality once issues have been resolved or deemed non-threatening.
- `registerOrUpdateClientChain(uint32 clientChainId, bytes32 peer, uint8 addressLength, string calldata name, string calldata metaInfo, string calldata signatureType)` : Enables the owner to register a new client chain or update an existing one's details such as its peer address and metadata. This is crucial for maintaining an accurate and functional network of client chains within the Exocore ecosystem.
- `setPeer(uint32 clientChainId, bytes32 clientChainGateway)` : Specifically allows the contract owner to set or update the LayerZero peer address for different client chains, which is necessary for the proper routing of cross-chain messages.
- `addOrUpdateWhitelistTokens(uint32 clientChainId, bytes32[] calldata tokens, uint8[] calldata decimals, uint256[] calldata tvlLimits, string[] calldata names, string[] calldata metaData)` : This function allows the owner to add new tokens to the whitelist or update existing ones, ensuring that only approved tokens are used within the network for operations like staking or rewards.

Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and execute a range of destructive actions. Specifically, they could disrupt normal operations by pausing and unpausing the contract at will, alter or falsify the registration of client chains, manipulate whitelisted tokens potentially introducing harmful elements into the ecosystem.

Additionally, the `ExocoreGateway` contract inherits the `OwnableUpgradeable` contract from OpenZeppelin, the owner has the following authorities within the contract:

- `renounceOwnership()` : Leaves the contract without owner;

- `transferOwnership()` : Transfers ownership of the contract to a new account.

---

In the contract `OAppCoreUpgradeable` the role `_owner` has authority over the functions shown in the list below.

- `setPeer(uint32 _eid, bytes32 _peer)` : This function allows the contract owner to set or update the peer address associated with a specific endpoint ID. The peer address represents another OApp instance that is trusted to send LayerZero messages to this OApp. Setting the peer to `bytes32(0)` effectively removes the peer address. This function is critical for managing communication channels between different OApp instances across various chains, ensuring that only authorized peers can interact.
- `setDelegate(address _delegate)` : This function allows the contract owner to set the delegate for the OApp, which is capable of making configuration changes inside the endpoint. The delegate has the authority to adjust settings directly on the LayerZero Endpoint, which is crucial for maintaining the operational flexibility and security of the OApp configurations.

Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and manipulate key functionalities of the contract, such as setting or updating peers across different chains and changing the delegate who manages configurations within the endpoint. This could lead to unauthorized cross-chain communications, misdirection of data or funds, and potential breaches in other connected contracts or systems.

Additionally, the `OAppCoreUpgradeable` contract inherits the `OwnableUpgradeable` contract from OpenZeppelin, the owner has the following authorities within the contract:

- `renounceOwnership()` : Leaves the contract without owner;
- `transferOwnership()` : Transfers ownership of the contract to a new account.

## Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

### Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND

- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

### Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

### Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR
- Remove the risky functionality.

# EGE-02 UNAUTHORIZED BOOTSTRAP REQUEST MAY LEAD TO DOS SITUATION

| Category | Severity | Location | Status |
|---|---|---|---|
| Access Control | Medium | src/core/ExocoreGateway.sol: 110 | Pending |

## Description

The `markBootstrapOnAllChains()` function sends bootstrap requests to client chains to initiate their bootstrapping and upgrades. It only sends the request if the `chainToBootstrapped` mapping status is `false`. After sending the request, the function updates the `chainToBootstrapped` mapping to `true` for each chain that received the request.

```
            if (!chainToBootstrapped[chainIndex]) {
                _sendInterchainMsg(chainIndex, Action.REQUEST_MARK_BOOTSTRAP, "",
true);
                // TODO: should this be marked only upon receiving a response?
                chainToBootstrapped[chainIndex] = true;
                emit BootstrapRequestSent(chainIndex);
            }
```

However, this function lacks access control, allowing any user to invoke it at any time. If a request is sent before the `spawnTime` of a client chain, the bootstrapping process for that chain will fail.

```
function markBootstrapped() public onlyCalledFromThis whenNotPaused {
        // whenNotPaused ensures that the upgrade does not proceed without
unpausing.
        // LZ checks made so far include:
        // lzReceive called by endpoint
        // correct address on remote (peer match)
        // chainId match
        // nonce match, requiring that inbound nonce is uint64(1).
        // TSS checks are not clear, but since it is not fully implemented yet, this
does not currently matter.
        if (block.timestamp < spawnTime) {
            revert Errors.BootstrapNotSpawnTime();
        }
...
```

Additionally, the `markBootstrapOnAllChains()` function does not consider the response from the client chain. It updates the `chainToBootstrapped` mapping to `true` once the request is sent, regardless of whether the request was successfully executed. As a result, Exocore gateway contract will not send the request to the failed client chain again, since the status is already set to `true`. Consequently, the failed client chain will never be successfully bootstrapped or upgraded.

## Recommendation

It is recommended to have access control on `markBootstrapOnAllChains` to ensure that all upgrades are deliberately decided upon by the Exocore project.

# BEN-01 | LACK OF INITIALIZATION OF OAPP FOR `Bootstrap`

| Category | Severity | Location | Status |
|---|---|---|---|
| Inconsistency | Minor | src/core/Bootstrap.sol: 61 | Pending |

## Description

The `Bootstrap` smart contract is structured within an inheritance hierarchy as follows: `Bootstrap` inherits from `BootstrapLzReceiver`, which in turn inherits from `OAppReceiverUpgradeable`, and finally from `OAppCoreUpgradeable`. The `OAppCoreUpgradeable` contract includes an initialization function named `__OAppCore_init`, which is responsible for setting the address authorized to configure the endpoint. However, the initialization function for the `Bootstrap` contract, termed `initialize()`, fails to call `__OAppCore_init()`. Although this oversight can be rectified by separately invoking `OAppCoreUpgradeable.setDelegate`, it is more efficient and coherent to integrate this step directly into the initial setup process.

## Recommendation

It is recommended to include the `OAppUpgradeable` initialization when initializing the bootstrap.

# BEN-02 | REGISTERED VALIDATORS ARE NOT SORTED

| Category | Severity | Location | Status |
|---|---|---|---|
| Inconsistency | Minor | src/core/Bootstrap.sol: 241 | Pending |

## Description

The description of the `registeredValidators` array states that it is an array sorted by voting power.

```
36 /// @notice This array stores the Ethereum addresses of all validators that have been registered in the contract.

37 /// These validators, sorted by their vote power, will be used to initialize the Exocore chain's validator set.

38 /// @dev A public array holding the Ethereum addresses of all validators that have been registered in the contract.

39     address[] public registeredValidators;
```

However, any user is able to call `registerValidator` to add their address to the end of this array, regardless of their voting power.

```
241         registeredValidators.push(msg.sender);
```

This means that the array might not be sorted based on the contract implementation, thus contrary to expectations.

## Recommendation

It is recommended to sort the array by voting power each time a new validator is registered or to no longer require the array to be sorted.

# BEN-03 | LOCKED BLOCKCHAIN NATIVE TOKENS

| Category | Severity | Location | Status |
|---|---|---|---|
| Inconsistency | Minor | src/core/Bootstrap.sol: 318~329, 358~369 | Pending |

## Description

The `Bootstrap` contract includes several functions marked as `payable`, allowing it to accept native tokens. However, it lacks a specific function to facilitate the withdrawal of these funds. Additionally, there's no implemented restriction to prevent the `msg.value` from being non-zero, potentially resulting in tokens being irreversibly locked within the contract.

```
318     function deposit(address token, uint256 amount)
```

```
358     function withdrawPrincipalFromExocore(address token, uint256 amount)
```

## Recommendation

It is advisable to enforce that the `msg.value` be zero in the `deposit` and `withdrawPrincipalFromExocore` functions of the `Bootstrap` contract.

# ECE-01 | UNREACHABLE FUNCTION

| Category | Severity | Location | Status |
|---|---|---|---|
| Logical Issue | Minor | src/core/ExoCapsule.sol: 273 | Pending |

## Description

The `withdrawNonBeaconChainETHBalance` function allows the `gateway` role to withdraw the `nonBeaconChainETHBalance`. According to the comment:

```
    /// @dev This function must be called through the gateway. @param amount must be greater than
```

This means the function can only be invoked through the gateway. However, there is no function in the `ClientChainGateway` contract that calls this function. Consequently, without a way to invoke it, the `nonBeaconChainETHBalance` will remain locked within the contract.

## Recommendation

It is recommended to add a function that calls this function to withdraw the `nonBeaconChainETHBalance`.

# ECE-02 | MISSING INITIALIZATION OF `ReentrancyGuardUpgradeable`

| Category | Severity | Location | Status |
|---|---|---|---|
| Coding Issue | Minor | src/core/ExoCapsule.sol: 151 | Pending |

## Description

The `ExoCapsule` smart contract extends the `ReentrancyGuardUpgradeable` contract, which includes a method called `__ReentrancyGuard_init()` designed to initialize the `status` variable. However, the initialization function for `ExoCapsule`, defined as `initialize()`, neglects to invoke `__ReentrancyGuard_init()`.

```
    function initialize(address gateway_, address capsuleOwner_, address
beaconOracle_) external initializer {
        require(gateway_ != address(0), "ExoCapsule: gateway address can not be
empty");
        require(capsuleOwner_ != address(0), "ExoCapsule: capsule owner address can
not be empty");
        require(beaconOracle_ != address(0), "ExoCapsule: beacon chain oracle
address should not be empty");

        gateway = INativeRestakingController(gateway_);
        beaconOracle = IBeaconChainOracle(beaconOracle_);
        capsuleOwner = capsuleOwner_;

        emit RestakingActivated(capsuleOwner);
    }
```

## Recommendation

It is recommended to ensure that all initialization functions from parent contracts are called within the initialization function of a derived contract. For instance, the `initialize` function of the `ExoCapsule` contract should include a call to `__ReentrancyGuard_init();` to properly set up all inherited functionalities.

# EGE-03 POTENTIAL IMPROPER REFUND ADDRESS HANDLING FOR FEE OVERPAYMENTS IN CROSS-CHAIN MESSAGES

| Category | Severity | Location | Status |
|---|---|---|---|
| Logical Issue | Minor | src/core/ExocoreGateway.sol: 604 | Pending |

## Description

The `_sendInterchainMsg` function in the `ExocoreGateway` contract is designed to handle the sending of interchain messages using the LayerZero protocol. This function is for enabling communications across different blockchain networks. The issue arises from how the function handles the refund of excess transaction fees. Specifically, the `msg.sender` is always used as the `refundAddress` in the `_lzSend` call.

```
593     function _sendInterchainMsg(uint32 srcChainId, Action act, bytes memory
 actionArgs, bool payByApp)
594         internal
595         whenNotPaused
596     {
597         bytes memory payload = abi.encodePacked(act, actionArgs);
598         bytes memory options = OptionsBuilder.newOptions().
addExecutorLzReceiveOption(
599             DESTINATION_GAS_LIMIT, DESTINATION_MSG_VALUE
600         ).addExecutorOrderedExecutionOption();
601         MessagingFee memory fee = _quote(srcChainId, payload, options, false);
602
603         MessagingReceipt memory receipt =
604         @>  _lzSend(srcChainId, payload, options, MessagingFee(fee.nativeFee, 0
), msg.sender, payByApp);
605         emit MessageSent(act, receipt.guid, receipt.nonce, receipt.fee.
nativeFee);
606     }
```

The `refundAddress` parameter in `_lzSend` determines where any excess funds (overpaid fees) are sent back after the transaction. In this case, it's set to `msg.sender`, which is the caller of the function. The `payByApp` boolean flag determines who pays for the transaction fees. If `payByApp` is `true`, it indicates that the contract itself (i.e., `ExocoreGateway`) should handle the fee payment.

When `payByApp` is `true`, the contract pays the transaction fees, but any excess fees are refunded to the `msg.sender` — not necessarily the entity that paid the fees (the contract). This could lead to a situation where the caller benefits from the refund of excess fees that they did not originally pay, for example, in the case of calling `markBootstrapOnAllChains` function.

## Recommendation

It's recommended to modify modify the `refundAddress` to be the contract address itself when `payByApp` is `true`. This ensures that any excess fees are returned to the payer (the contract), maintaining financial consistency. For example:

```
        MessagingReceipt memory receipt =
            _lzSend(srcChainId, payload, options, MessagingFee(fee.nativeFee, 0
), payByApp ? address(this) : msg.sender, payByApp);
```

# EGE-05 | UNRESOLVED TODO COMMENT

| Category | Severity | Location | Status |
|---|---|---|---|
| Coding Style, Logical Issue | Minor | src/core/ExocoreGateway.sol: 119 | Pending |

## Description

In the `ExocoreGateway` contract, the `markBootstrapOnAllChains` function is tasked with initiating the bootstrapping process on client chains by sending a `REQUEST_MARK_BOOTSTRAP` message. The process ensures that each client chain is marked as bootstrapped following these steps:

## ExocoreGateway Contract

1. Transmit the `REQUEST_MARK_BOOTSTRAP` message to the respective client chain.
2. Subsequently, record that the client chain has been bootstrapped.

## Bootstrap Contract

1. The `lzReceive` function is activated by the LayerZero endpoint.
2. This triggers the `markBootstrapped` function to conclude the bootstrapping process.

It is important to note that the `markBootstrapped` function might fail for various reasons, such as the current time not yet reaching the defined spawn time or the `clientChainGatewayLogic` not being established. Despite potential failures in the `Bootstrap` contract, the `ExocoreGateway` still records the client chain as bootstrapped.

A `TODO` comment within the code highlights this issue, suggesting a need for implementation to ensure that a client chain is marked as bootstrapped only after receiving a successful response:

```
    function markBootstrapOnAllChains() public whenNotPaused nonReentrant {
        ...
        for (uint256 i = 0; i < chainIndices.length; ++i) {
            uint32 chainIndex = chainIndices[i];
            if (!chainToBootstrapped[chainIndex]) {
                _sendInterchainMsg(chainIndex, Action.REQUEST_MARK_BOOTSTRAP, "", true);
            @>  // TODO: should this be marked only upon receiving a response?
                chainToBootstrapped[chainIndex] = true;
                emit BootstrapRequestSent(chainIndex);
            }
        }
    }
```

## Recommendation

It's recommended to implement the `TODO` logic in the `markBootstrapOnAllChains` function to ensure that a client chain is only marked as bootstrapped after successfully receiving a confirmation response. This adjustment would enhance the reliability of the bootstrapping process, ensuring that the status of each client chain accurately reflects the successful completion of all necessary operations before it is considered fully bootstrapped.

# ENB-03 | LACK OF STORAGE GAP OR NAMESPACED STORAGE LAYOUT IN UPGRADEABLE CONTRACT

| Category | Severity | Location | Status |
|---|---|---|---|
| Design Issue | Minor | src/core/BaseRestakingController.sol: 21; src/core/BootstrapLzReceiver.sol: 14; src/core/ClientGatewayLzReceiver.sol: 16; src/core/LSTRestakingController.sol: 16; src/core/NativeRestakingController.sol: 22; src/lzApp/OAppCoreUpgradeable.sol: 12; src/lzApp/OAppReceiverUpgradeable.sol: 12; src/lzApp/OAppSenderUpgradeable.sol: 17; src/lzApp/OAppUpgradeable.sol: 19 | Pending |

## Description

When updating upgradeable smart contracts for new features or bug fixes, keeping the state variables' declaration order unchanged is essential to avoid storage layout issues.

A practical solution is to include unused state variables or explicitly named storage gaps (like `__gap`) in the base contracts. This foresight allows reserved slots for future use, ensuring that any additions to the contract's state won't disrupt the storage pattern of derived contracts or the compatibility with previously deployed versions. After `ERC-7201`, it is also possible to place all storage variables of a contract into one or more structs like `Namespaced Storage Layout`.

The problem of "Lack of Storage Gap Or NameSpaced Storage Layout in Upgradeable Contract" occurs when **these storage gaps are not incorporated into the base contract's logic nor the base contract defines the namespace storage layout**. As a result, if new state variables are added to the base contract, they might overwrite existing variables in the child contracts due to storage slot collisions.

**In the current contract, the contract allows for future upgrades and is also inherited by other contracts. However, the storage gap is missing for the the current contract, nor is the `namespaced storage layout` used.**

For detailed guidelines and best practices, refer to the following OpenZeppelin documentation:

- https://docs.openzeppelin.com/contracts/4.x/upgradeable#storage_gaps
- https://docs.openzeppelin.com/upgrades-plugins/1.x/writing-upgradeable#storage-gaps

## Recommendation

To mitigate this issue:

1. For enhanced flexibility in future upgrades of the logic contract, it is prudent to reserve a storage gap of an appropriate size in the base contract. This is achieved by declaring a fixed-size array, typically of `uint256` elements, each occupying a 32-byte slot, in the base contract. Label this array with the identifier `__gap` or any name prefixed with `__gap_` to indicate its purpose as a reserved space clearly.

2. it is also possible by placing all storage variables of a contract into one or more structs like `Namespaced Storage Layout` .

More detailed info :

- https://docs.openzeppelin.com/contracts/4.x/upgradeable#storage_gaps

# OAS-02 | POTENTIAL UNINTENDED NATIVE TOKEN TRANSFER

| Category | Severity | Location | Status |
|---|---|---|---|
| Logical Issue | Minor | src/lzApp/OAppSenderUpgradeable.sol: 90~91 | Pending |

## Description

According to the comment in the `_lzSend` function, when `byApp` is set to `true`, the caller is not required to specify `msg.value` for paying the native fee. In this case, the application itself handles the payment of the native fee. The `_payNative` function supports this behavior. However, it does not check if `msg.value` is zero when `byApp` is `true`.

As a result, users might mistakenly send native tokens via `msg.value` even though it is unnecessary when `byApp` is `true`. These tokens could end up being locked in the contract, leading to potential financial losses for the user.

## Recommendation

It is recommended to add a check within the `_payNative` function to ensure that no native tokens are sent by the user when `byApp` is `true`. If any native tokens are detected, the transaction should be reverted to avoid locking the funds.

# VEN-01 | INCOMPATIBILITY WITH DEFLATIONARY TOKENS

| Category | Severity | Location | Status |
|---|---|---|---|
| Logical Issue | Minor | src/core/Vault.sol: 77, 78, 79 | Pending |

## Description

When transferring deflationary ERC20 tokens, the input amount may not be equal to the received amount due to the charged transaction fee. For example, if a user sends 100 deflationary tokens (with a 10% transaction fee), only 90 tokens actually arrive in the contract. However, a failure to discount such fees may allow the same user to withdraw 100 tokens from the contract, which causes the contract to lose 10 tokens in such a transaction.

```
78        underlyingToken.safeTransferFrom(depositor, address(this), amount);
```

- Transferring tokens by `amount`.

```
79        totalDepositedPrincipalAmount[depositor] += amount;
```

- The `amount` appears to be used for bookkeeping purposes without compensating the potential transfer fees.

## Recommendation

We advise the client to regulate the set of tokens supported and add necessary mitigation mechanisms to keep track of accurate balances if there is a need to support deflationary tokens.

# BEN-04 IMPROPER USE OF `msg.sender` IN `_delegateTo` FUNCTION OF `Bootstrap` CONTRACT

| Category | Severity | Location | Status |
|---|---|---|---|
| Coding Style | Informational | src/core/Bootstrap.sol: 450 | Pending |

## Description

In the `_delegateTo` function of the `Bootstrap` contract, this function is designed to support the delegation process where users assign tokens to validators. As an internal function, it's designed to be invoked by other functions within the contract that manage related delegation activities.

```
450         uint256 withdrawable = withdrawableAmounts[msg.sender][token];
```

The line above references `msg.sender` to determine the account balance available for delegation. This approach could lead to issues if the `msg.sender` differs from the intended `user` parameter.

## Recommendation

To correct this inconsistency and ensure that the function accurately checks and updates the withdrawable balance for the correct user, it should utilize the `user` parameter that is passed to the function. For example:

```
450         uint256 withdrawable = withdrawableAmounts[user][token];
```

# EGE-04 DISCUSSION ON PROCESSING `REQUEST_DEPOSIT` REQUEST IN `ExocoreGateway`

| Category | Severity | Location | Status |
|---|---|---|---|
| Design Issue, Inconsistency | Discussion | src/core/ExocoreGateway.sol: 358~361, 511~514 | Pending |

## Description

In the `ExocoreGateway` contract, the `lzReceive` function, which is invoked by the LayerZero endpoint of the Exocore chain, is responsible for processing incoming requests from client chains. When dealing with the `REQUEST_DEPOSIT` message, if the `ASSETS_CONTRACT` precompile returns false, the `requestDeposit` and `requestDepositThenDelegateTo` functions will immediately revert and fail to send a `RESPOND` message back to the client chain indicating the failure. This behavior is inconsistent with the processing of other types of requests such as `REQUEST_WITHDRAW_PRINCIPAL_FROM_EXOCORE` and `REQUEST_WITHDRAW_REWARD_FROM_EXOCORE`, where a response is guaranteed to be sent back to the client chain, regardless of the success or failure of the precompile contract execution.

```
351     function requestDeposit(uint32 srcChainId, uint64 lzNonce, bytes calldata
 payload) public onlyCalledFromThis {
352         ...
353
354         (bool success, uint256 updatedBalance) = ASSETS_CONTRACT.depositTo(
srcChainId, token, depositor, amount);
355         if (!success) {
356         @>  revert DepositRequestShouldNotFail(srcChainId, lzNonce);
357         }
358
359         _sendInterchainMsg(srcChainId, Action.RESPOND, abi.encodePacked(lzNonce
, success, updatedBalance), true);
360
361         emit DepositResult(true, bytes32(token), bytes32(depositor), amount);
362     }
```

```
    function requestDepositThenDelegateTo(uint32 srcChainId, uint64 lzNonce, bytes calldata payload)
        public
        onlyCalledFromThis
    {
        ...
        bool result = false;
        (bool success, uint256 updatedBalance) = ASSETS_CONTRACT.depositTo(srcChainId, token, depositor, amount);
        if (!success) {
        @>  revert DepositRequestShouldNotFail(srcChainId, lzNonce);
        }
        emit DepositResult(true, bytes32(token), bytes32(depositor), amount);

        ...
    }
```

If the precompile encounters internal errors during deposit operations, the principal amounts for stakers on client chains remain unchanged. The audit team is seeking clarification on whether this behavior is by design in the current implementation.

# GSE-01 | DISCUSSION ON INABILITY TO REMOVE WHITELISTED TOKENS

| Category | Severity | Location | Status |
|---|---|---|---|
| Design Issue | Discussion | src/storage/GatewayStorage.sol: 18 | Pending |

## Description

In the event a whitelisted token is compromised or has large changes to its functionality, the Exocore project may no longer wish to support the token. The auditing team would like to discuss whether or not removing whitelisted tokens should be an option.

# NRC-01 | DISCUSSION ON PARTIAL WITHDRAWAL REQUEST OMISSION IN `processBeaconChainWithdrawal` FUNCTION

| Category | Severity | Location | Status |
|---|---|---|---|
| Logical Issue | Discussion | src/core/NativeRestakingController.sol: 122~123 | Pending |

## Description

The `processBeaconChainWithdrawal` function verifies a withdrawal proof from the beacon chain and forwards the information to the Exocore chain. This function only sends a request to the Exocore chain when a full withdrawal is requested, meaning the `partialWithdrawal` flag is false. However, if the withdrawal is partial, no request is sent to the Exocore chain.

We would like to discuss this behavior with the team to ensure that this design is intentional and correct.

# OAS-01 DISCUSSION ON `_lzSend` FUNCTION IN `OAppSenderUpgradeable` CONTRACT

| Category | Severity | Location | Status |
|---|---|---|---|
| Design Issue | Discussion | src/lzApp/OAppSenderUpgradeable.sol: 91~93, 137 | Pending |

## Description

In the `_lzSend` function of the `OAppSenderUpgradeable` contract, the `lzTokenFee` is consistently paid by `msg.sender` if supported. Although the current implementation within `Exocore` project does not utilize `lzTokenFee` for paying message fees, it's important to note that `OAppSenderUpgradeable` serves as an abstract contract, and future applications of `_lzSend` might involve using it for fee payments. The auditing team seeks clarification on the following points:

1. When the contract is supposed to pay the fees (i.e., when `byApp` is true), the current implementation of `_lzSend` still processes the transfer of LayerZero tokens from `msg.sender` to the endpoint. If `msg.sender` is not the contract itself, then it means the `msg.sender` is covering the message fees. Is this design intentional?
2. When `byApp` is set to true, indicating that the OApp should pay the LayerZero token fees, and if `msg.sender` is the contract address, the contract should perform a `transferFrom` to the endpoint. However, there is no logic present to approve the necessary allowance, which means the contract does not authorize itself for a specific amount beforehand.

```
    function _lzSend(
        uint32 _dstEid,
        bytes memory _message,
        bytes memory _options,
        MessagingFee memory _fee,
        address _refundAddress,
        bool byApp
    ) internal virtual returns (MessagingReceipt memory receipt) {
// @dev Push corresponding fees to the endpoint, any excess is sent back to the
_refundAddress from the
        // endpoint.
        uint256 messageValue = _payNative(_fee.nativeFee, byApp);
        if (_fee.lzTokenFee > 0) {
        @>  _payLzToken(_fee.lzTokenFee);
        }
```

```
    function _payLzToken(uint256 _lzTokenFee) internal virtual {

// @dev Cannot cache the token because it is not immutable in the endpoint.
        address lzToken = endpoint.lzToken();
        if (lzToken == address(0)) {
            revert LzTokenUnavailable();
        }

        // Pay LZ token fee by sending tokens to the endpoint.
    @>  IERC20(lzToken).safeTransferFrom(msg.sender, address(endpoint),
_lzTokenFee);
    }
```

# VEN-02 | DISCUSSION ON SOURCE OF VAULT REWARDS

| Category | Severity | Location | Status |
|---|---|---|---|
| Design Issue | Discussion | src/core/Vault.sol: 111 | Pending |

## Description

The auditing team would like to understand the source of rewards for vault contracts. Any rewards acquired by a user are added to their withdrawable balance and the reward token is the same as the deposit token. This can lead to a situation where the rewards a user withdraws are actually the deposits of other users. As such, we would like to know how sufficient rewards are guaranteed to exist within the contract.

# OPTIMIZATIONS | EXOCORE (SMART CONTRACTS)

| ID | Title | Category | Severity | Status |
|---|---|---|---|---|
| ENB-01 | Inefficient Memory Parameter | Inconsistency | Optimization | Pending |

# ENB-01 | INEFFICIENT MEMORY PARAMETER

| Category | Severity | Location | Status |
|---|---|---|---|
| Inconsistency | Optimization | src/core/ClientChainGateway.sol: 118; src/core/CustomProxyAdmin.sol: 35, 35~46; src/core/ExocoreGateway.sol: 609; src/interfaces/IClientChainGateway.sol: 15; src/interfaces/IExocoreGateway.sol: 20 | Pending |

## Description

One or more parameters with `memory` data location are never modified in their functions and those functions are never called internally within the contract. Thus, their data location can be changed to `calldata` to avoid the gas consumption copying from calldata to memory.

```
15      function quote(bytes memory _message) external view returns (uint256
 nativeFee);
```

`quote` has memory location parameters: `_message`.

```
20      function quote(uint32 srcChainid, bytes memory _message) external view
returns (uint256 nativeFee);
```

`quote` has memory location parameters: `_message`.

---

```
35      function changeImplementation(address proxy, address implementation, bytes
memory data) public virtual {
```

`changeImplementation` has memory location parameters: `data`.

---

```
 609      function quote(uint32 srcChainid, bytes memory _message) external view
returns (uint256 nativeFee);
```

`quote` has memory location parameters: `_message`.

---

```
 118      function quote(bytes memory _message) external view returns (uint256
 nativeFee);
```

`quote` has memory location parameters: `_message`.

## Recommendation

We recommend changing the parameter's data location to `calldata` to save gas.

- For Solidity versions prior to 0.6.9, since public functions are not allowed to have calldata parameters, the function visibility also needs to be changed to `external`.
- For Solidity versions prior to 0.5.0, since parameter data location is implicit, changing the function visibility to `external` will change the parameter's data location to calldata as well.

# APPENDIX | EXOCORE (SMART CONTRACTS)

## Finding Categories

| Categories | Description |
|---|---|
| Coding Style | Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable. |
| Coding Issue | Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues. |
| Access Control | Access Control findings are about security vulnerabilities that make protected assets unsafe. |
| Inconsistency | Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification. |
| Logical Issue | Logical Issue findings indicate general implementation issues related to the program logic. |
| Centralization | Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code. |
| Design Issue | Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories. |

## Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux "sha256sum" command against the target file.

# DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

CertiK | Securing the Web3 World

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is a leading blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.